

机器学习

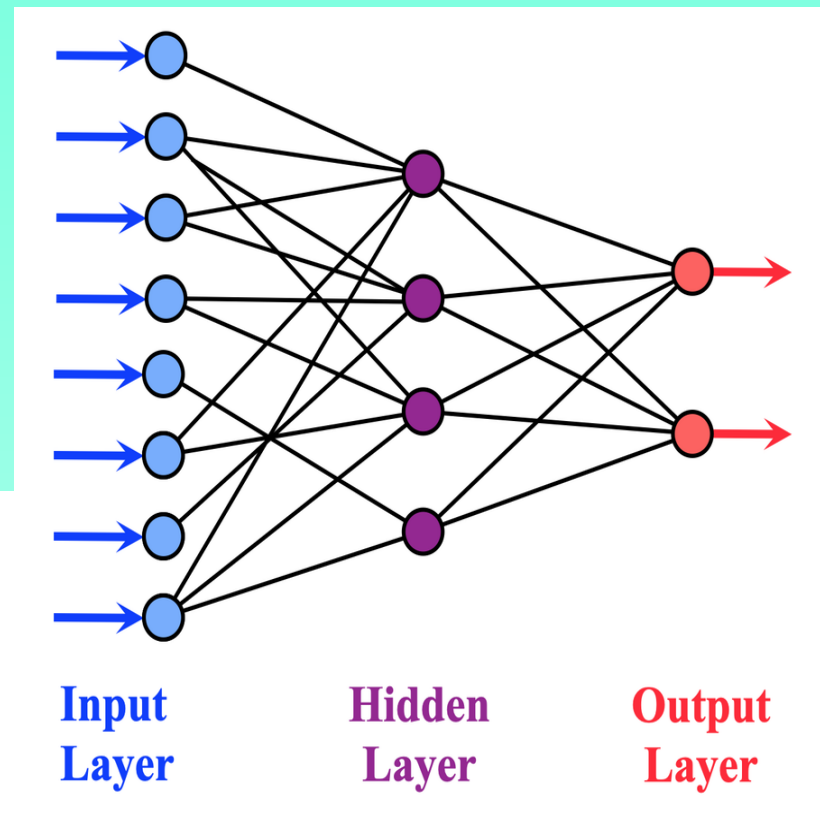
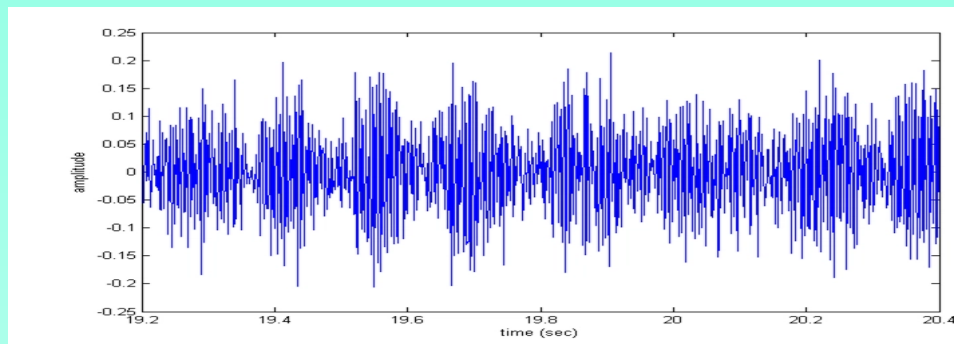
第四章 人工神经网络 (ANN)

主要内容

- 神经元的主要类型：感知器、线性单元、sigmoid 单元
- 训练单个神经元的学习算法：感知器学习算法
- 神经网络的表示
- 多层前馈网络及学习算法：**BP**算法及其推导
- 两种不同的梯度下降学习技术
- 问题讨论
 - ANNs 的表示能力
 - 假设空间搜索及归纳偏置
 - 过拟合问题
 - BP算法的变体
- **BP**网络应用实例

4.1 简介

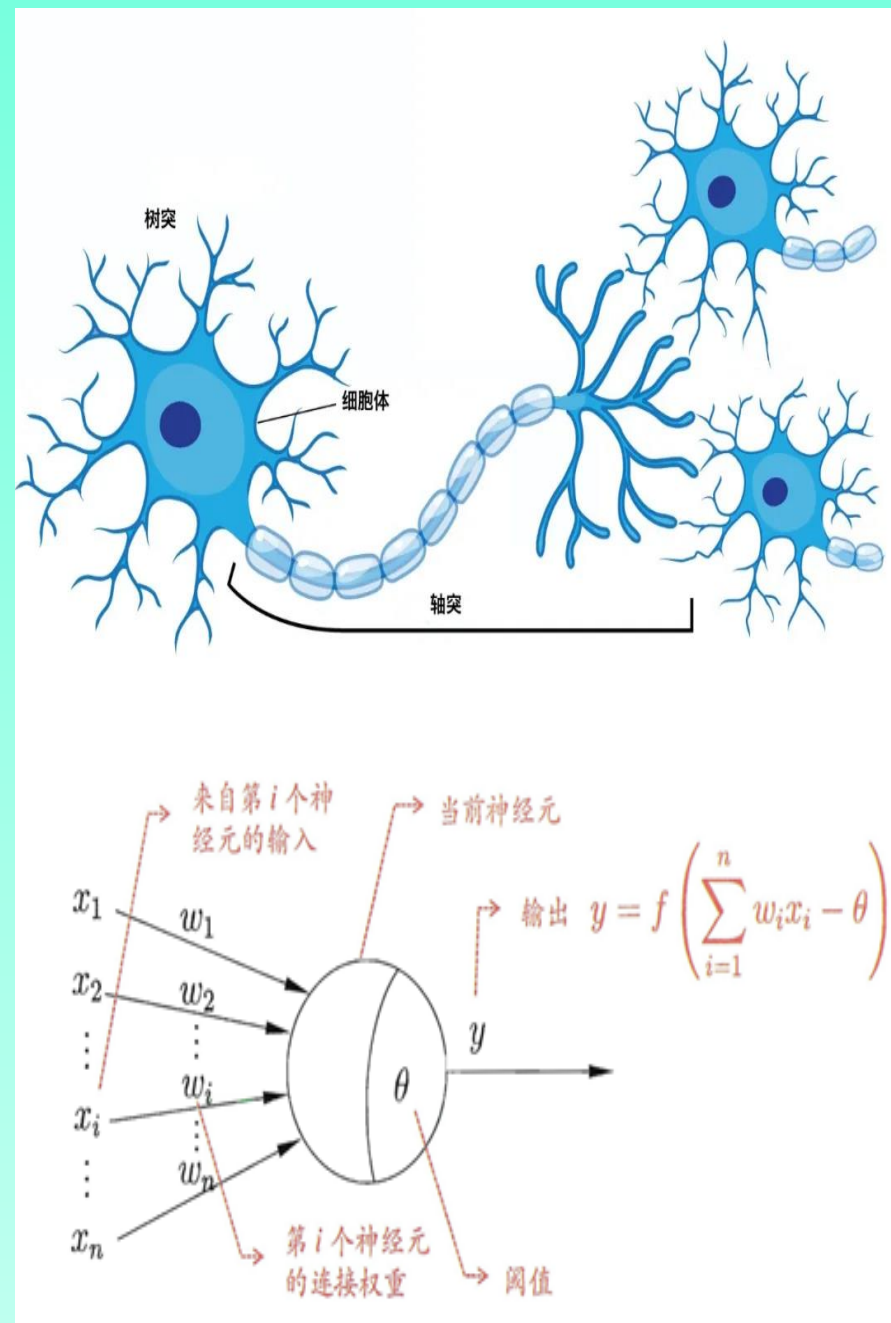
- 人工神经网络提供了学习实值、离散值和向量值函数的方法
 - 对于某些问题，例如学习、解释复杂的现实世界的传感器数据，神经网络非常有效



- BP算法在许多领域被广泛应用

4.1.1 生物学动机

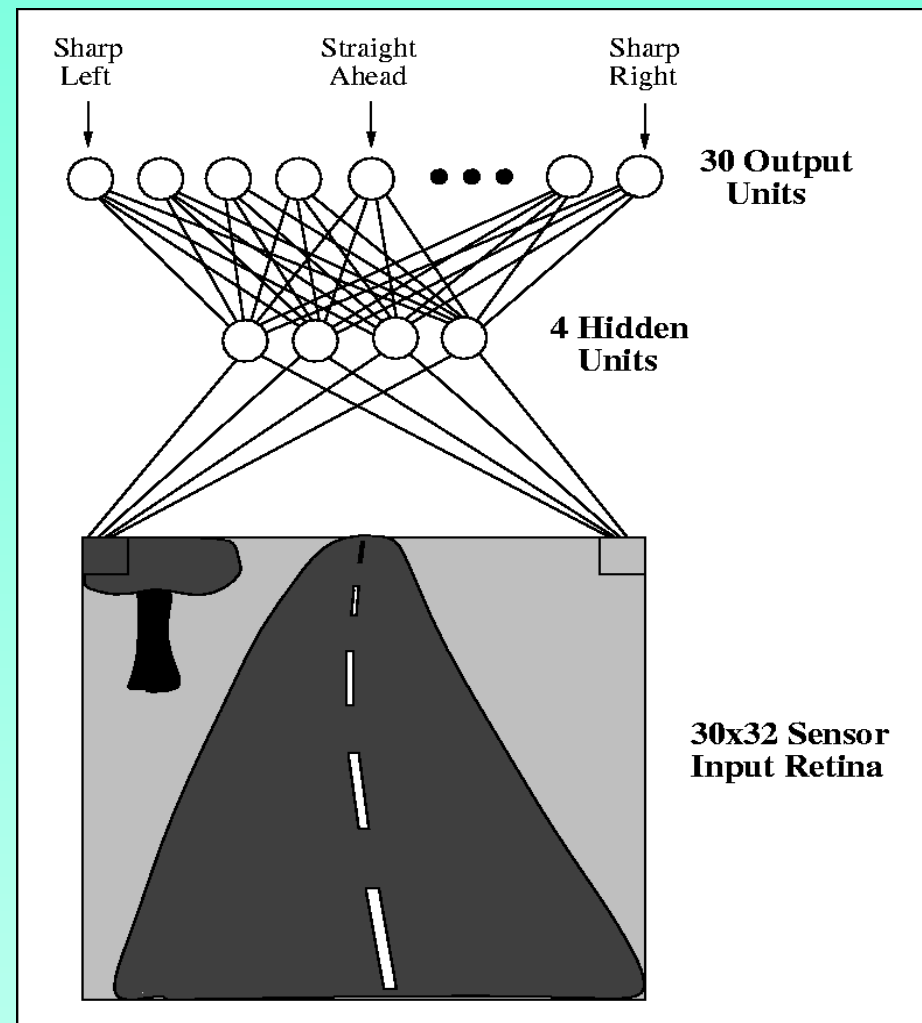
- 生物学习系统是由相互连接的神经元组成的非常复杂的网络构成的。
 - 人类的大脑是一个紧密相连的网络(10^{11} 神经元, 每一个链接其他 10^4)
 - 神经元的活动通常通过与其他神经元的连接而被激活或抑制
- 生物神经网络的信息处理能力
 - 与计算机速度相比, 神经元的切换时间较慢。然而人类能够快速做出复杂的决定(为什么?)
 - 基于分布式表示的高度并行计算
- 人工神经网络系统就是为了捕捉这种能力而设计的



- 事实上，神经网络只是在一定程度上受到生物神经系统的启发
- ANN 的两类研究团体
 - 利用神经网络来研究和模拟生物学习过程
 - 获得高效的机器学习算法，而不管这些算法是否反映了生物过程
- 本书中，我们讨论后者

4.2 神经网络表示

- ALVINN (Pomerleau 1993)
 - 自动驾驶
 - 输入: 30x32 图片
 - 从下方进入节点的实线是它的输入
 - 隐藏单元
 - 它们的输出仅在网络内可用，不能作为网络输出的一部分
 - 输出单元
 - 每个输出单元对应一个特定的驾驶方向，输出值决定选择哪个驾驶动作



ALVINN

ALVINN drives 70 mph on highways

相机照片

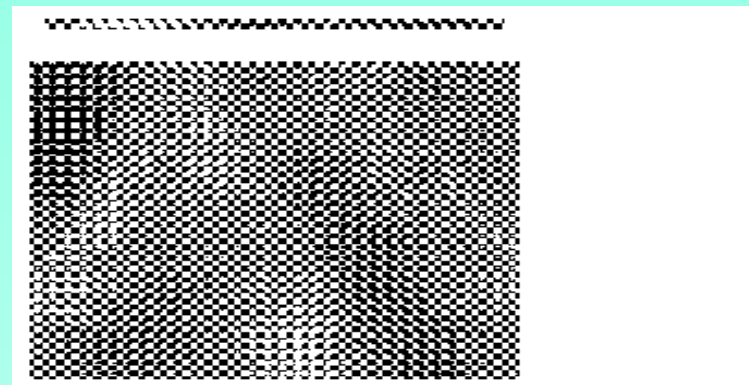
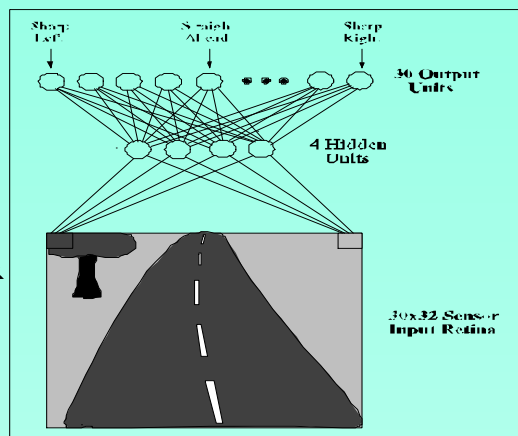


30个输出单元

进入4个隐藏层单元的
30x32个权值

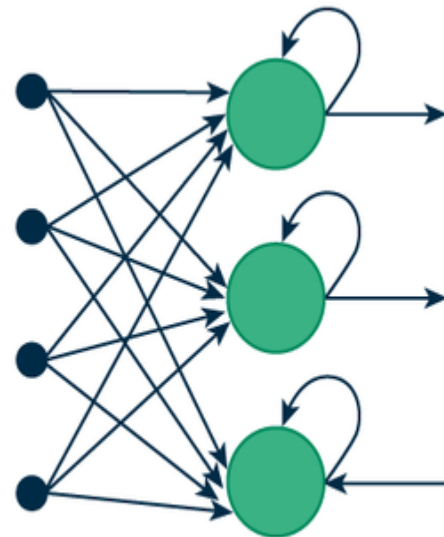
4 个隐藏
层单元

30x32 像素
作为输入

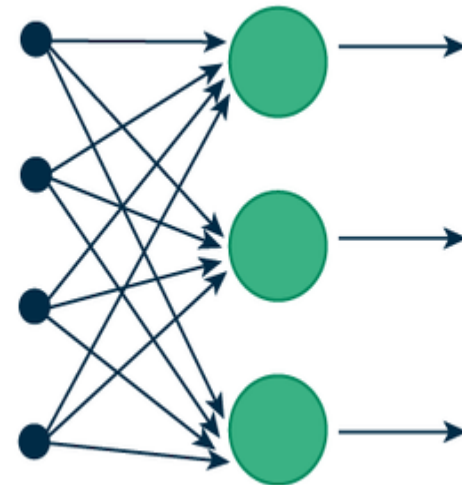


在公路上时速70英里

- ALVINN 的网络结构
 - 单元分层互联，形成有向无环图。
- ANN的结构类型
 - 无环或有环
 - 有向或无向
- 主要研究BP神经网络
 - 结构:一个有向图，可能包含环
 - 学习:为网络中的每条边选取一个权值

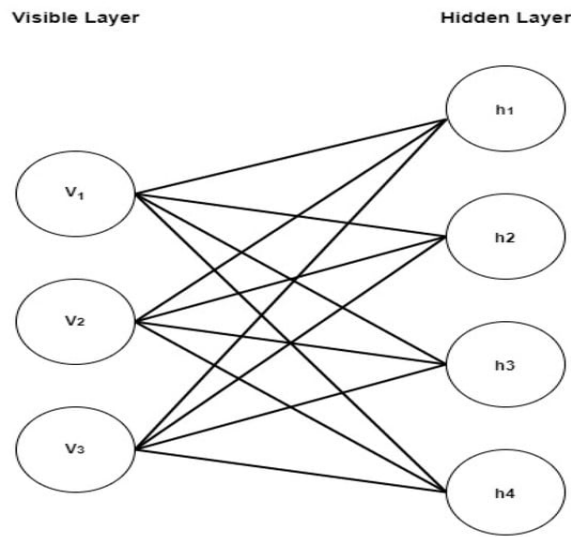


(a) Recurrent Neural Network



(b) Feed-Forward Neural Network

Restricted Boltzmann machine



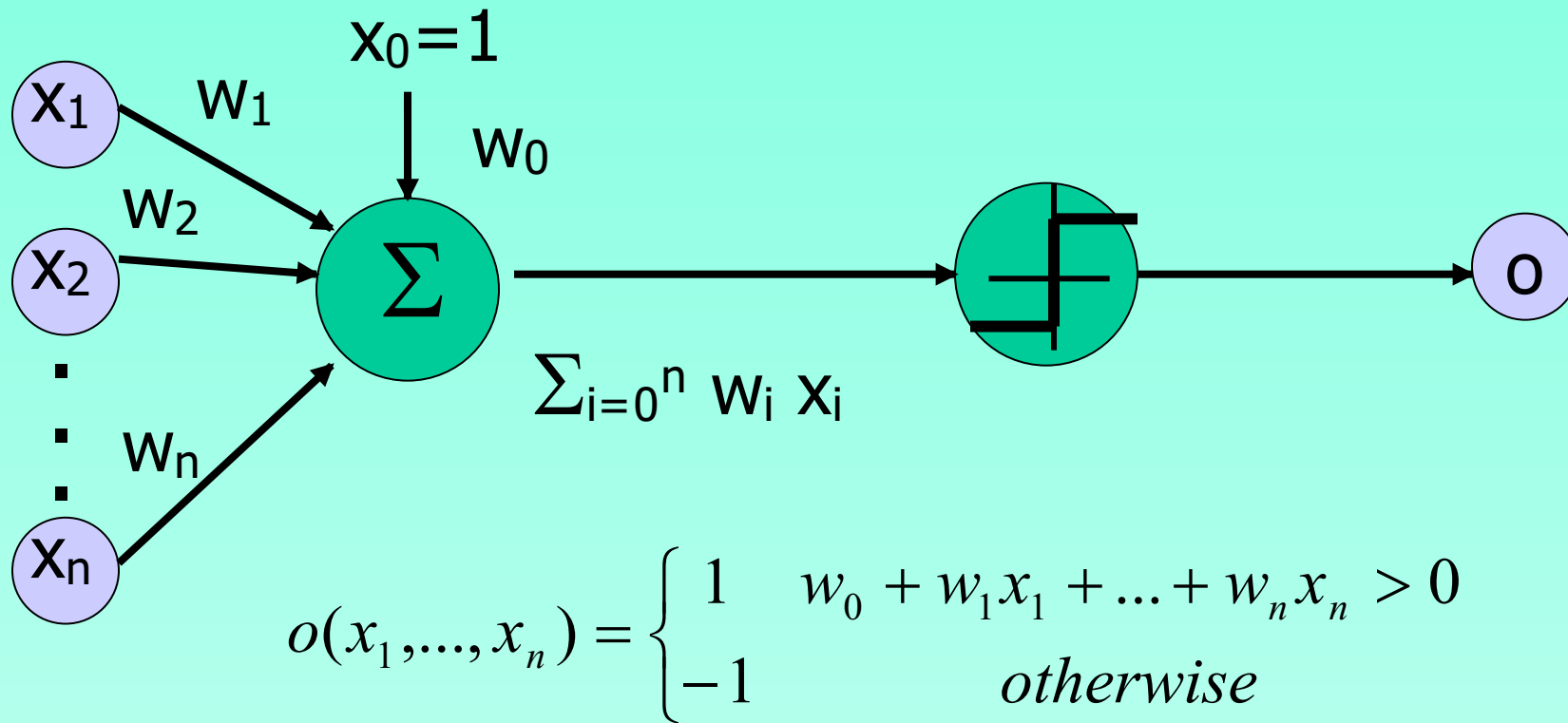
4.3 适合神经网络学习的问题

- 具有以下特征的问题
 - 实例由许多属性-值对表示(结构化数据)
 - 向量描述实例，高维
 - 也适用非结构化数据（深度学习）
 - 目标函数:离散值函数、实值函数或向量函数
 - 训练数据可能包含错误
 - 有噪声的、复杂的传感器数据(摄像头、麦克风)

- 可能需要快速求出目标函数值
 - 可容忍长时间的训练
 - 人类能否理解学习到的目标函数并不重要
-
- **BP**是最常用的神经网络学习算法

4.4 感知器

- 线性阈值单元(LTU)



- 感知器函数

$$o(\vec{x}) = \text{sgn}(\vec{w} \cdot \vec{x})$$

$$\text{sgn}(y) = \begin{cases} 1 & y > 0 \\ -1 & \text{otherwise} \end{cases}$$

- 感知器学习中的假设空间

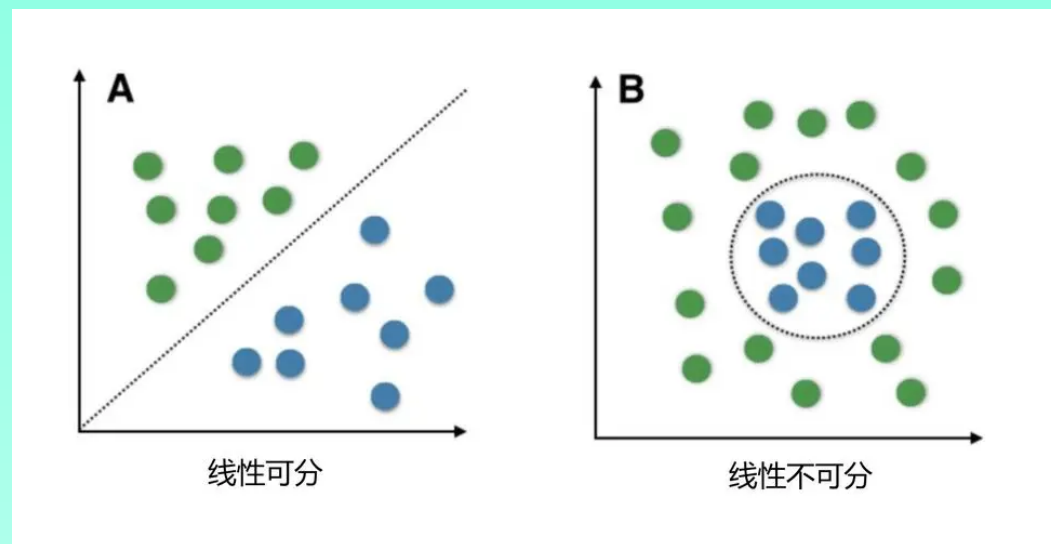
$$H = \{\vec{w} \mid \vec{w} \in R^{n+1}\}$$

4.4.1 感知器的表征能力

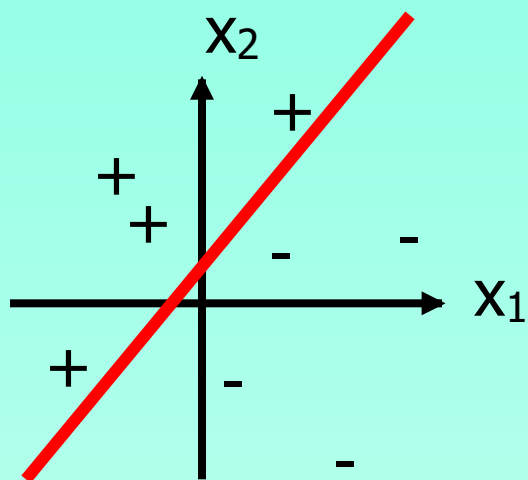
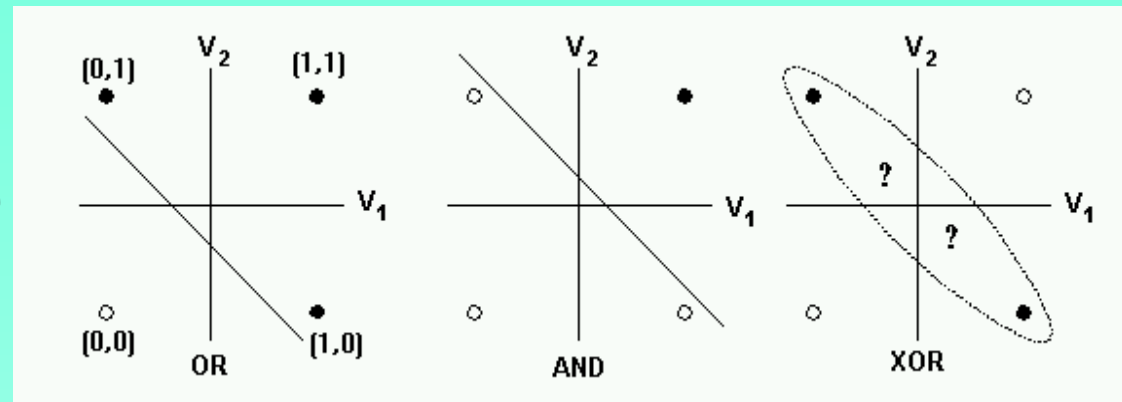
- 感知器可以看作是n维实例空间中的一个超平面决策面
 - 超平面一侧为正例
 - 反例在另一边
- 判定超平面的方程

$$\vec{w} \cdot \vec{x} = 0$$

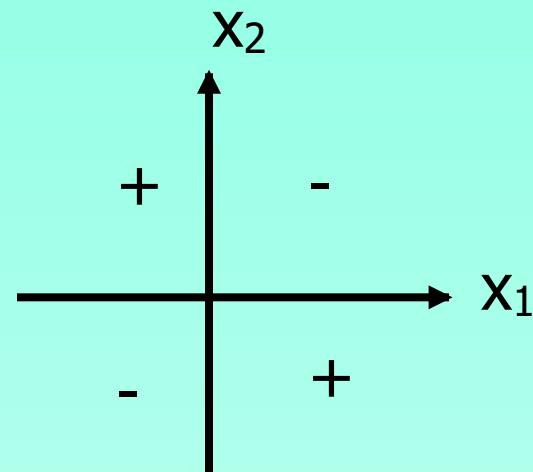
- 线性可分的数据集



- 单个感知器的表征能力
 - 布尔函数:m-of-n函数(与、或逻辑函数)
 - AND OR NAND NOR
 - 不能表示非线性可分的函数(例如异或)

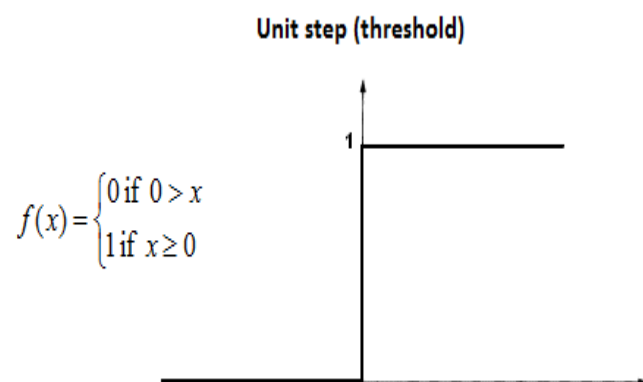
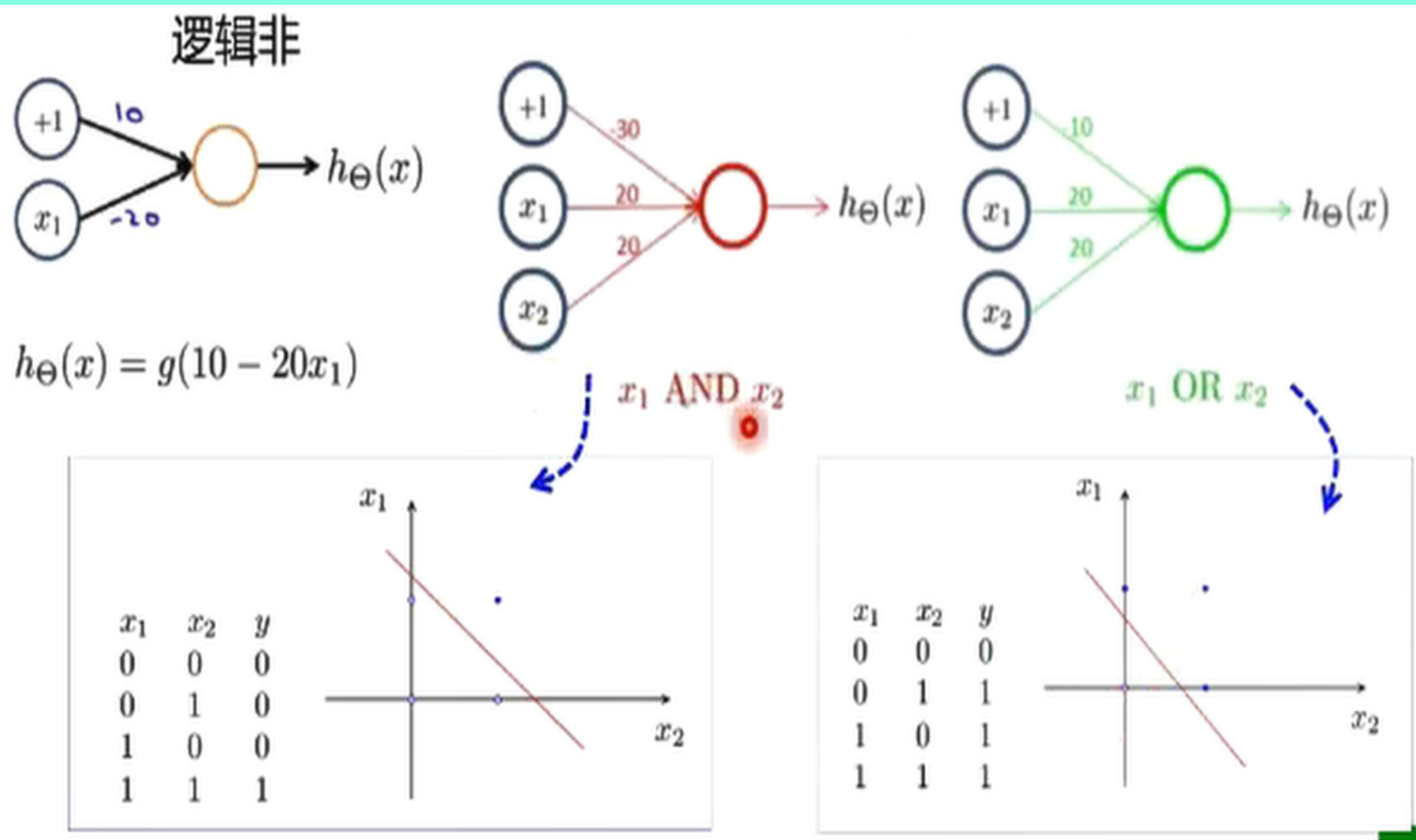


and(x_1, x_2) 选择权重 $w_0 = -1.5$, $w_1 = 1$, $w_2 = 1$



异或

感知器的表征能力示例



- 感知器网络的表示能力

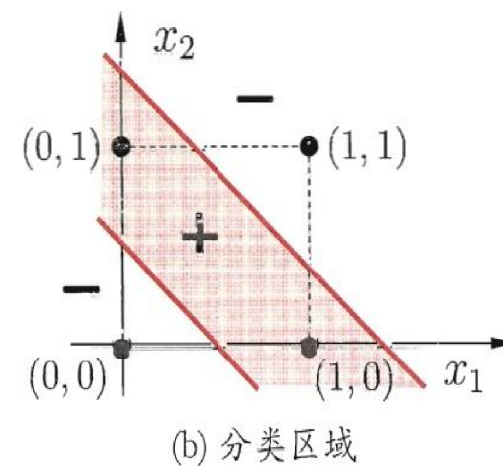
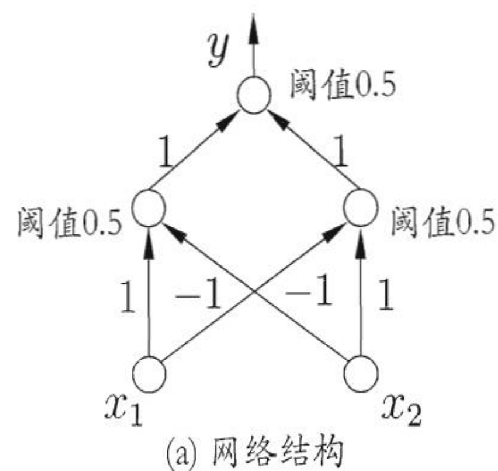
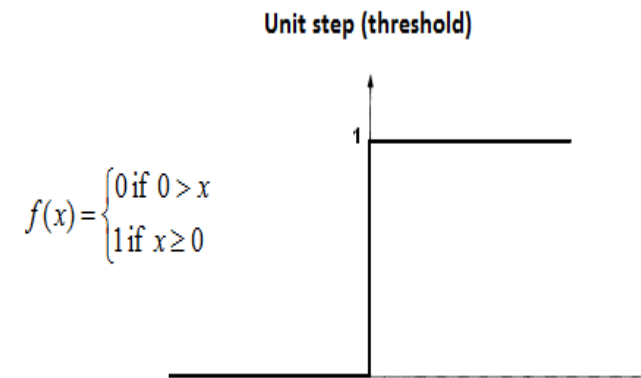
- 基于与、或、与非、或非，感知器网络可表示大量布尔函数

- 只用两层感知器网络就可表示所有布尔函数

- 一种方式

- 用析取范式表示布尔函数

- $(x_1 \wedge x_2 \wedge x_3) \vee (x_3 \wedge x_4 \wedge x_5)$



能解决异或问题的两层感知机

4.4.2 感知器训练法则

- 学习单个感知器的权重

$$o(x_1, \dots, x_n) = \begin{cases} 1 & w_0 + w_1x_1 + \dots + w_nx_n > 0 \\ -1 & otherwise \end{cases}$$

- 学习问题
 - 确定权重向量，使感知器对给定训练样例输出正确的1或-1
- 学习算法
 - 感知器法则
 - **Delta** 法则
 - 两者都收敛到一个可接受的权向量
 - 多层网络学习算法的基础

感知器训练法则

$$w_i = w_i + \Delta w_i$$

$$\Delta w_i = \eta (t - o) x_i$$

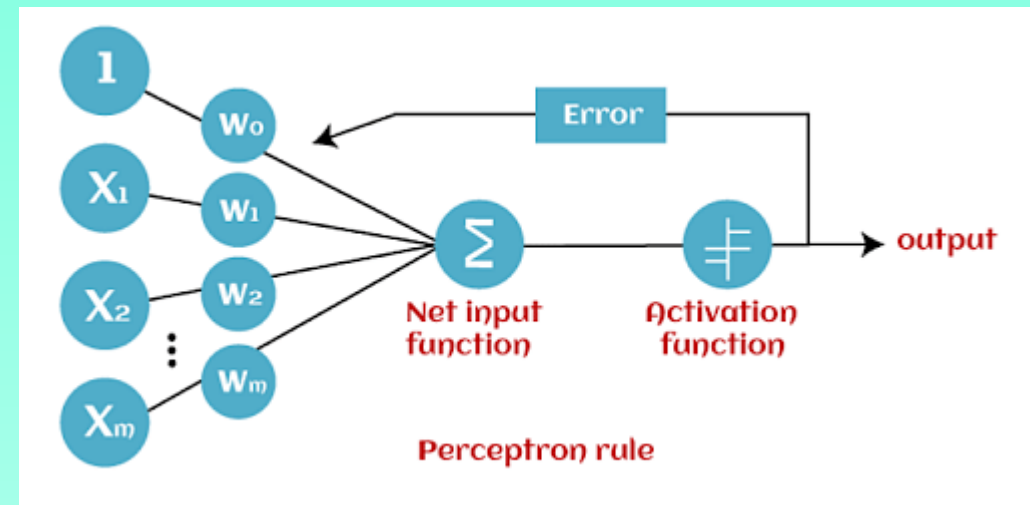
$t=c(x)$: 目标值

o : 感知器输出

Error: $(t - o)$

η : 一个很小的常数 (e.g. 0.1), 叫做学习率

- 如果输出是正确的($t=o$), w_i 不会改变
- 如果输出是错误的($t \neq o$), w_i 会改变
 - 使感知器的输出更加接近 t (?)
- 学习算法的收敛条件
 - 训练数据线性可分
 - η 足够小



$$(x,t) = ([2,1], -1)$$

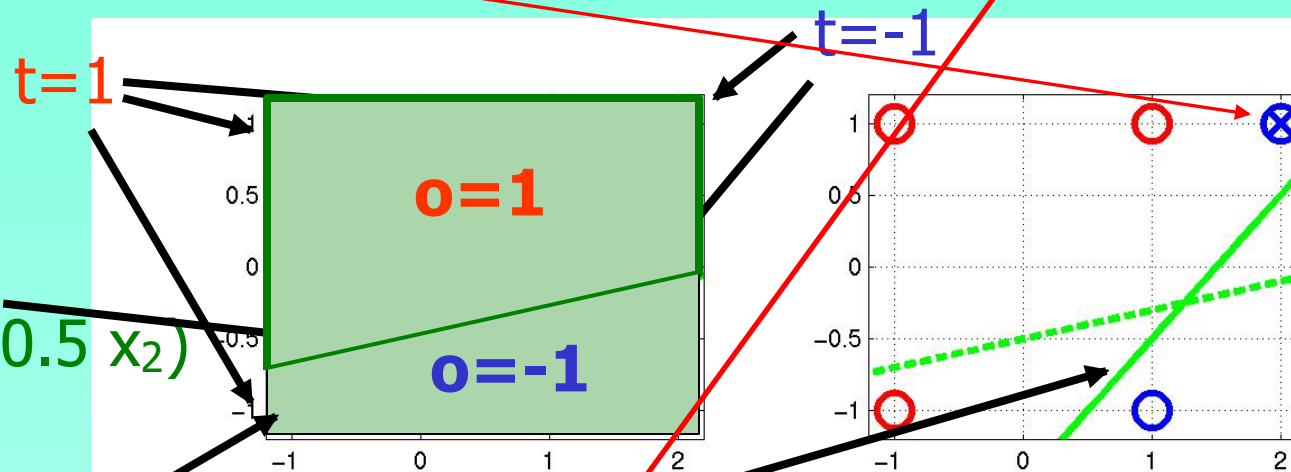
$$o = \text{sgn}(0.45 - 2 * 0.3 + 1 * 0.3) = 1$$

$$(x,t) = ([1,1], 1)$$

$$o = \text{sgn}(0.25 - 1 * 0.7 + 1 * 0.1) = -1$$

$$w0 = [0.25 \ -0.1 \ 0.5]$$

$$f(x) = \text{sgn}(0.25 - 0.1 x_1 + 0.5 x_2)$$



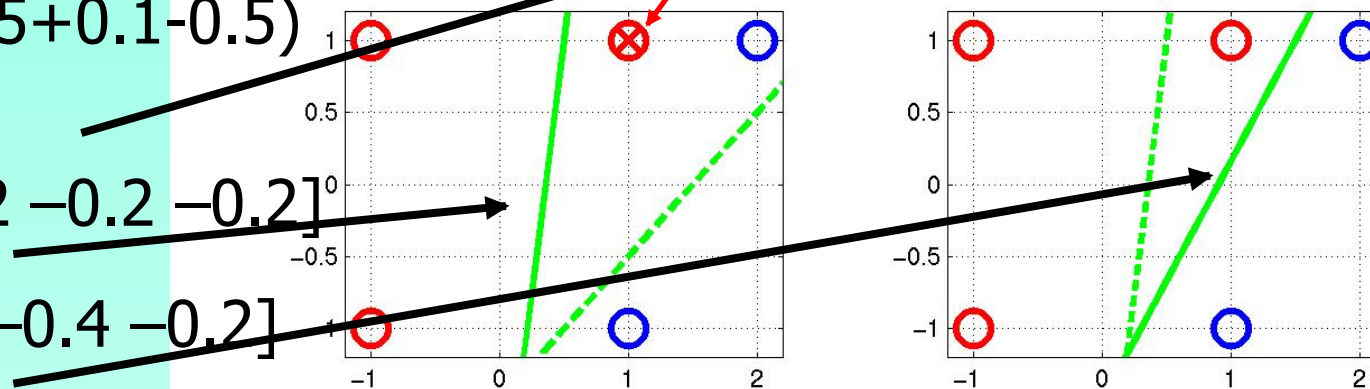
$$(x,t) = ([-1,-1], 1)$$

$$o = \text{sgn}(0.25 + 0.1 - 0.5) = -1$$

$$\Delta w1 = [0.2 \ -0.2 \ -0.2]$$

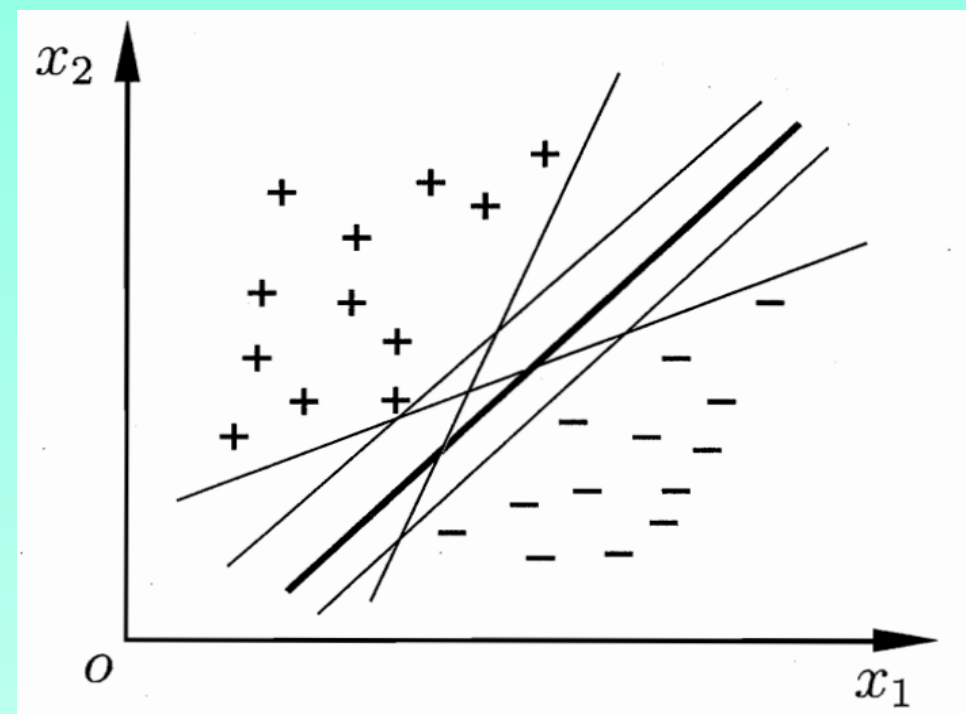
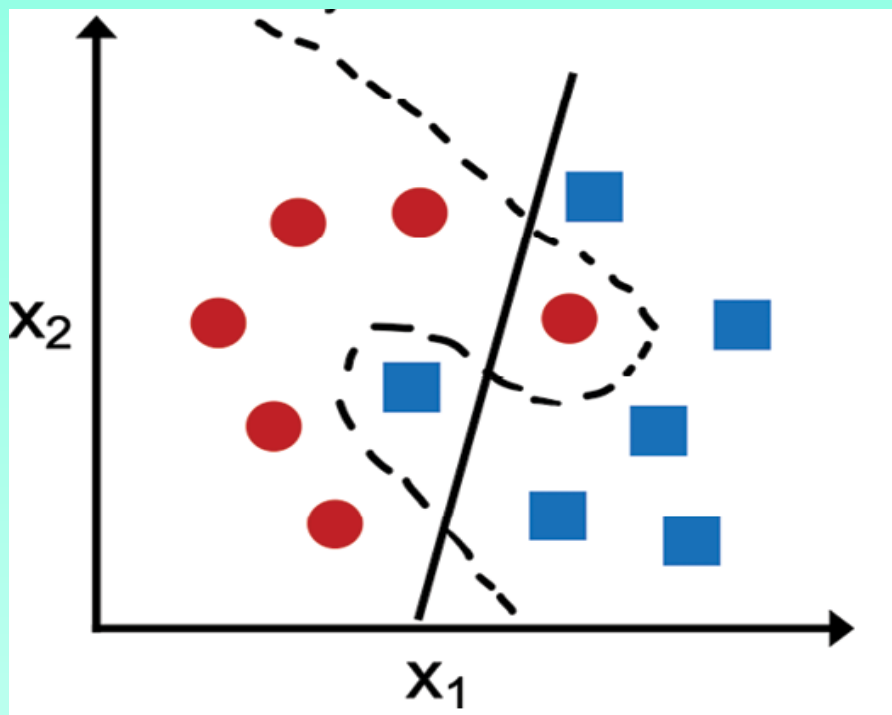
$$\Delta w2 = [-0.2 \ -0.4 \ -0.2]$$

$$\Delta w3 = [0.2 \ 0.2 \ 0.2]$$



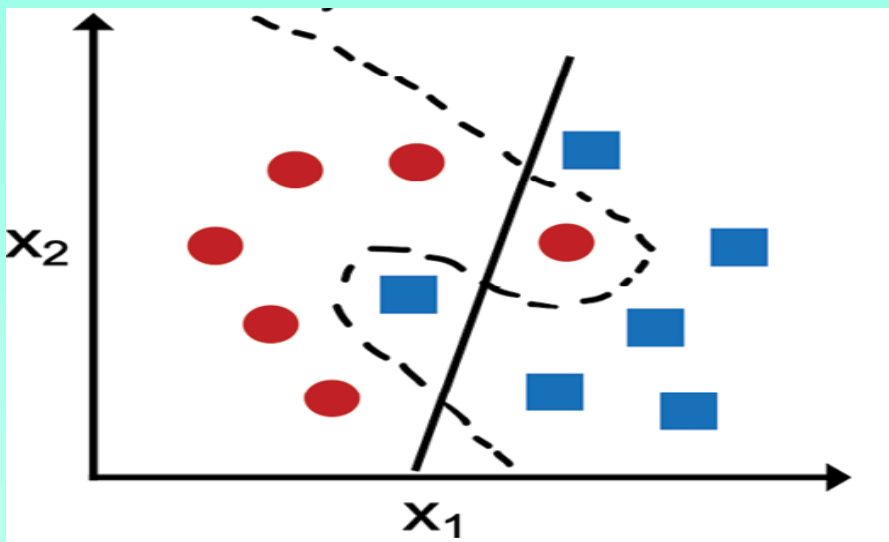
– 收敛条件 (Minsky & Papert 1969)

- 训练样例是线性可分的, η 足够小
- 如果数据不是线性可分的, 不能保证收敛



4.4.3 梯度下降和Delta 法则

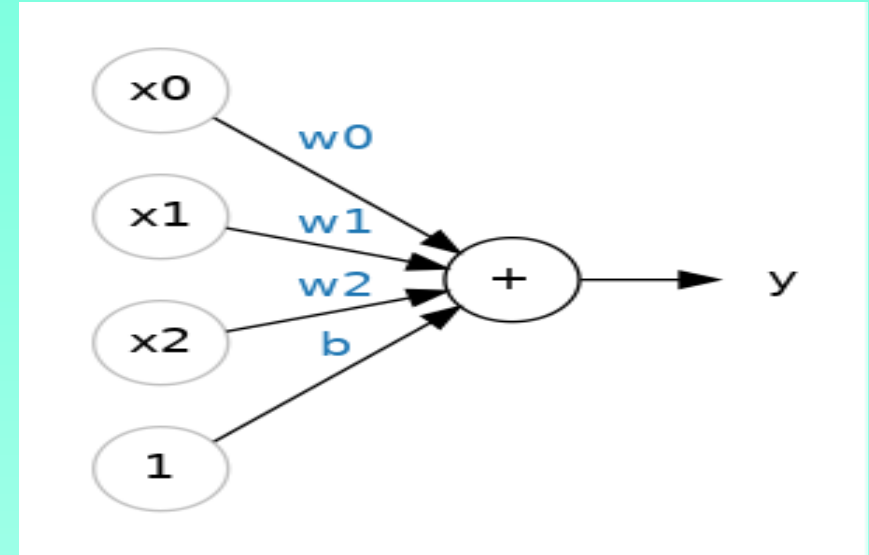
- Delta 法则克服了感知器规则的不足
 - 如果训练样例不是线性可分的，**delta**法则会收敛到目标概念的最佳近似
- Delta法则的关键思想
 - 使用梯度下降法搜索H，以找到最适合训练实例的权值



- 理解 Delta 法则

- Delta法则的任务:训练一个无阈值感知器

$$o(\vec{x}) = \vec{w} \cdot \vec{x}$$



- 指定一个度量标准,评估假设(权重向量)相对于训练样例的训练误差

$$\vec{w} = - \sum_{\epsilon} -$$

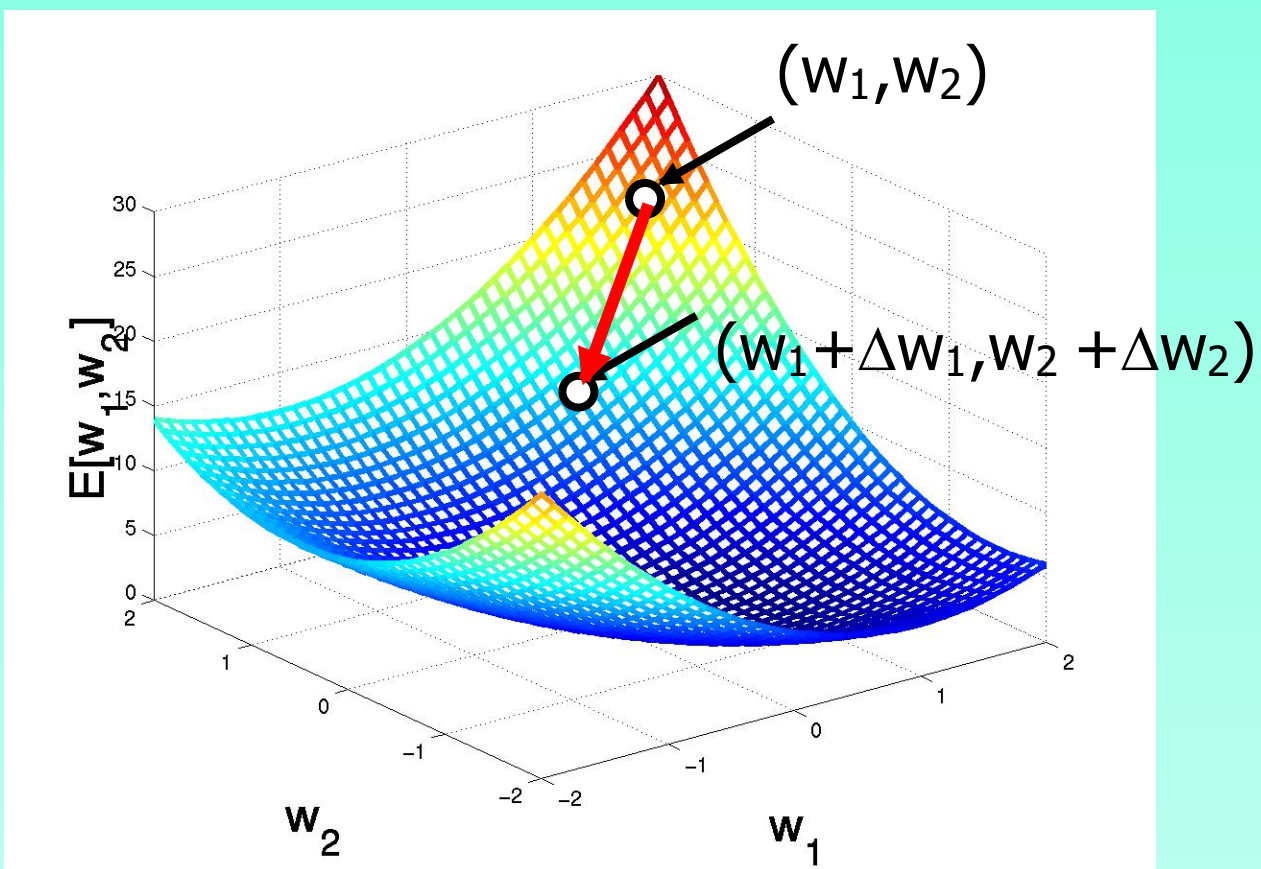
- 找到使E最小的 $\vec{w}$

- 第6章给出了一种基于贝叶斯理论的证明,在特定的条件下,对于给定的训练数据,使E最小化的假设就是H中最可能的假设

4.4.3.1 可视化假设空间

• 图 4-4

- 假设空间 H 由 (w_1, w_2) 组成。对于线性单元，误差面有一个全局最小值



• 梯度下降搜索

- 确定使 E 最小的权值向量

- 从任意的初始权值向量 w 开始
- 沿误差面下降最陡峭的方向改变 w
- 重复修改 w ，直到达到全局最小误差

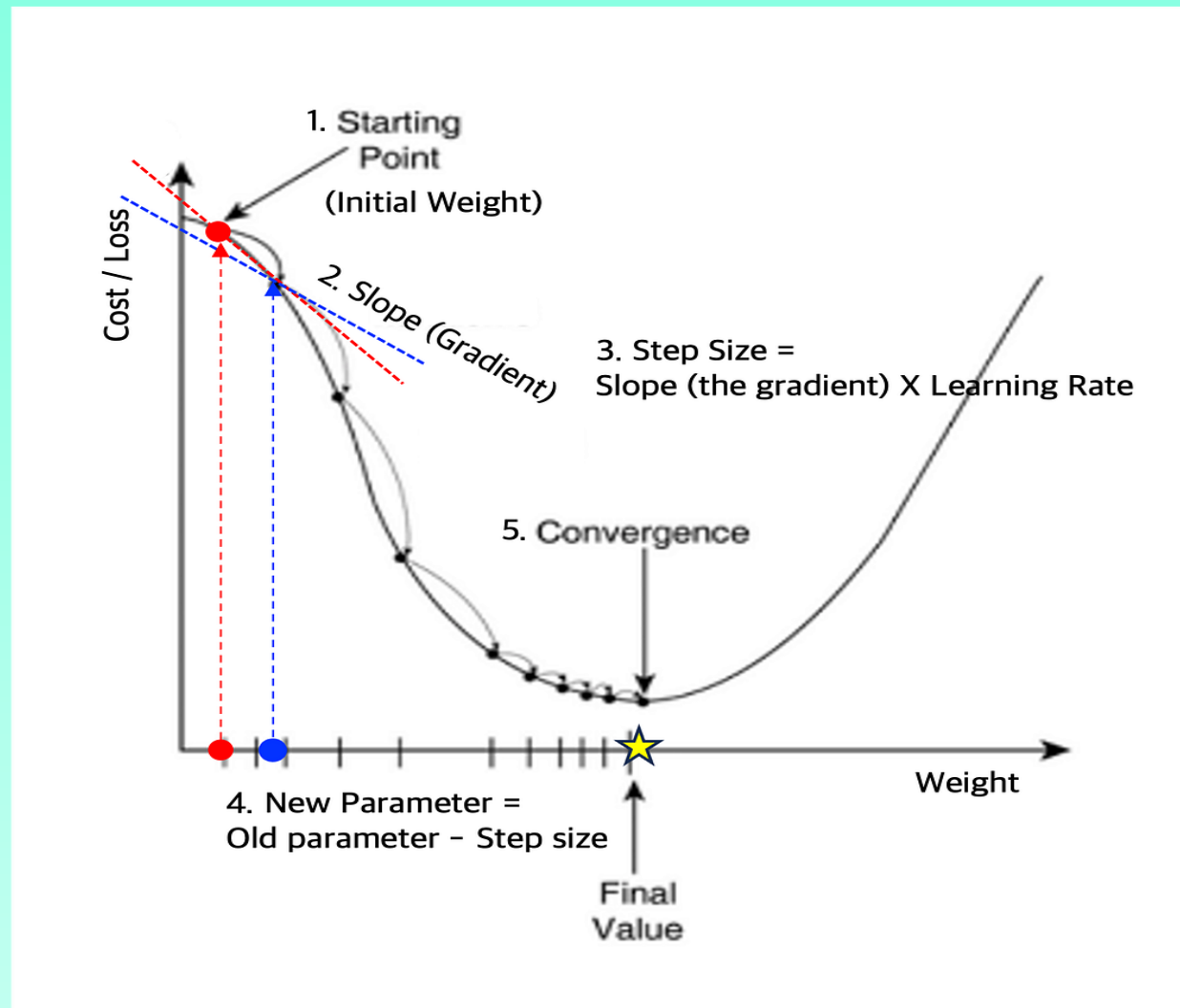
4.4.3.2 梯度下降法则的推导

- 怎样找到最陡的下降方向？
 - 计算E关于向量 $\vec{w}$ 的各个分量的导数, 写作 $\nabla E(\vec{w})$
 - 梯度指定E的最快增长方向
 - 这个向量的负方向就是下降最快的方向

- 梯度下降训练规则

$$\vec{w} \leftarrow \vec{w} + \Delta \vec{w}$$

$$\Delta \vec{w} = -\eta \nabla E(\vec{w})$$



梯度下降

$$D=\{<(1,1),1>,<(-1,-1),1>,<(1,-1),-1>,<(-1,1),-1>\}$$

Gradient:

$$\nabla E[w]=[\partial E/\partial w_1,\dots \partial E/\partial w_n]$$

$$\Delta w=-\eta \nabla E[w]$$

$$\Delta w_i=-\eta \partial E/\partial w_i$$

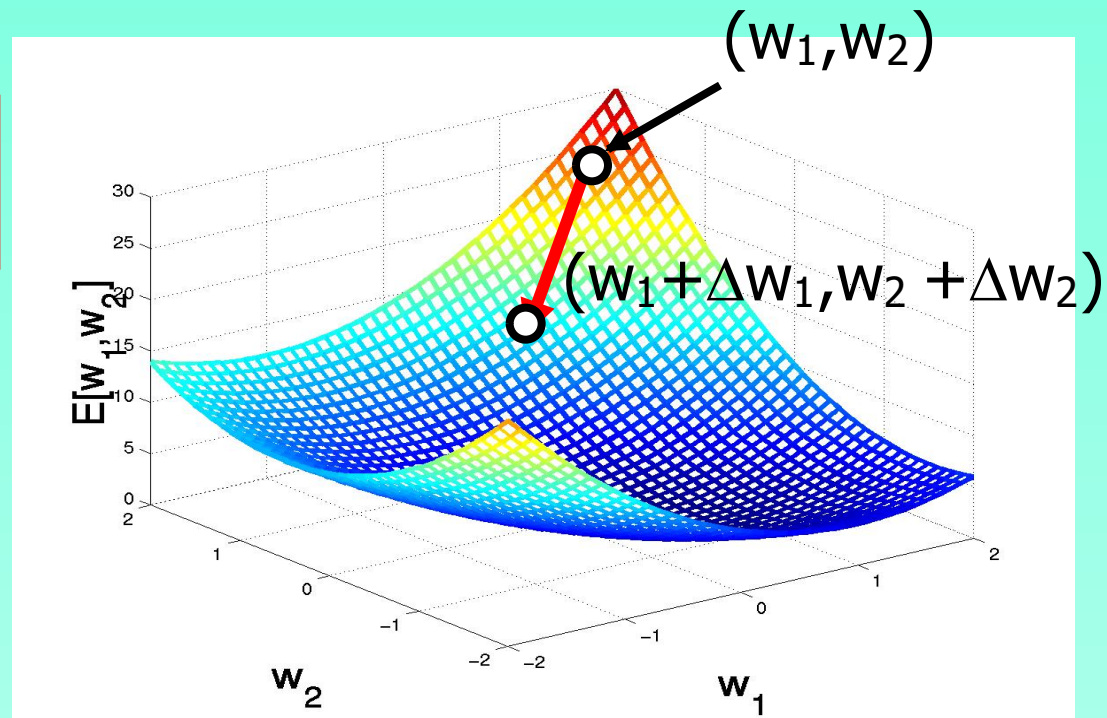
$$= -\eta \partial (1/2 \sum_d (t_d - o_d)^2) / \partial w_i$$

$$= -\eta 1/2 \sum_d \partial ((t_d - o_d)^2) / \partial w_i$$

$$= -\eta 1/2 \sum_d \partial ((t_d - o_d)^2) / \partial o_d * (\partial o_d / \partial w_i)$$

$$= \eta \sum_d (t_d - o_d)(x_{id})$$

注: $o_d = \sum w_i x_{id}$



梯度下降(Batch Gradient Descent)

Gradient-Descent(*training_examples*, η)

- Training_examples 中每个训练样例形式为序偶 $((x_1, \dots, x_n), t)$ 其中 $(x_1, \dots, x_n)$ 是输入值向量， t 是目标输出值， η 是学习率
- 初始化每个 w_i 为一个很小的随机值
- 在满足终止条件之前，我们执行
 - 初始化每个 Δw_i 为0
 - 对每一个 在训练样例中的序偶 $((x_1, \dots, x_n), t)$ ，执行
 - 输入实例 $(x_1, \dots, x_n)$ 到线性单元并计算输出 o
 - 对线性单元的每一个权 w_i ，执行
 - $\Delta w_i = \Delta w_i + \eta (t - o) x_i$
 - 对线性单元的每一个 w_i ，执行
 - $w_i = w_i + \Delta w_i$

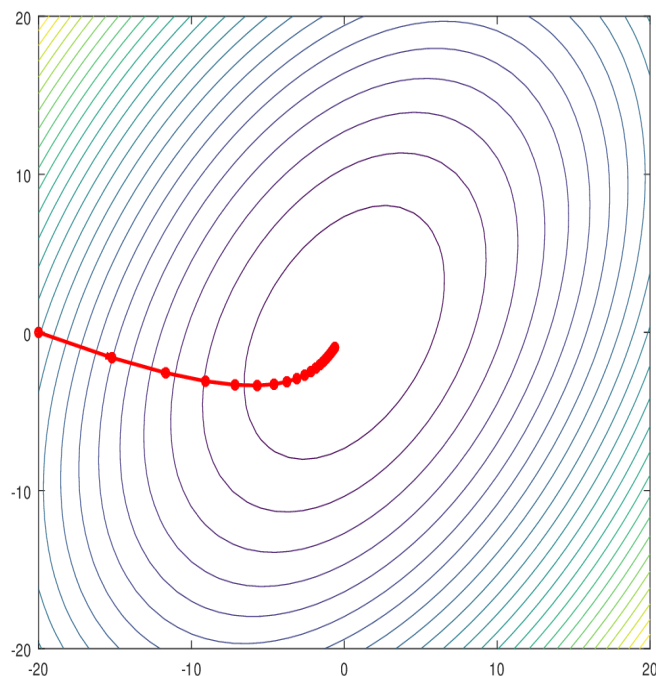
循环累加

- 收敛条件

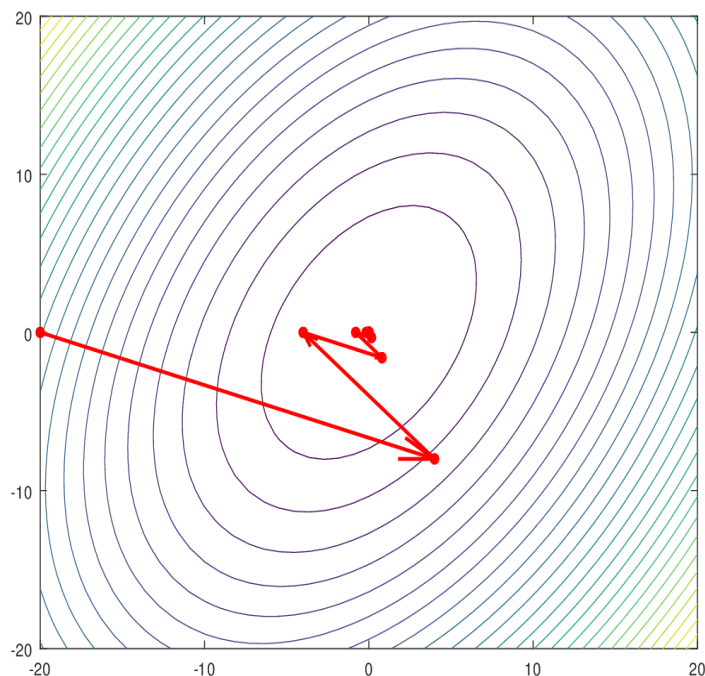
- 由于误差面只包含一个单一的全局最小值，无论训练样例是否线性可分，给定一个足够小的学习速率，该算法将收敛到一个具有最小误差的权向量 $\vec{w}$

- 对算法的修改

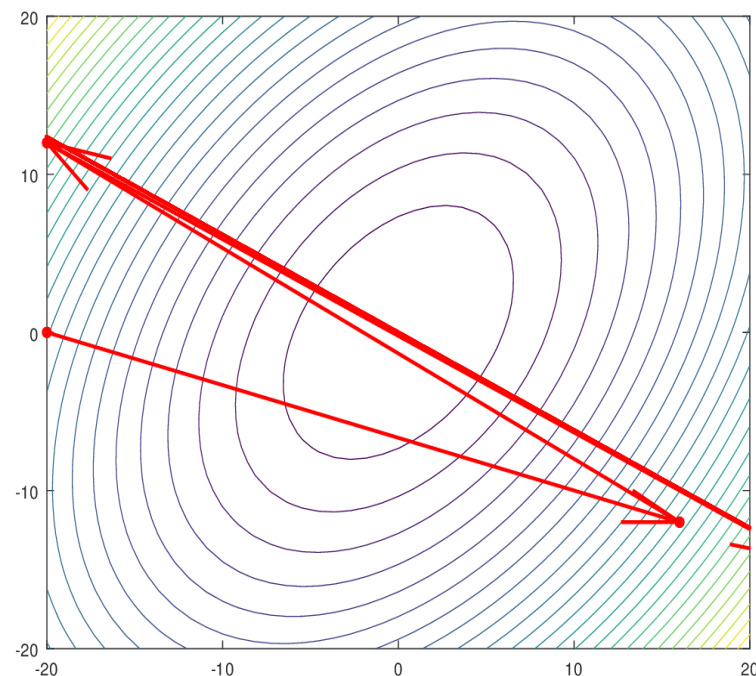
- 随着梯度下降次数的增加，学习速率的值逐渐降低



足够小学习率



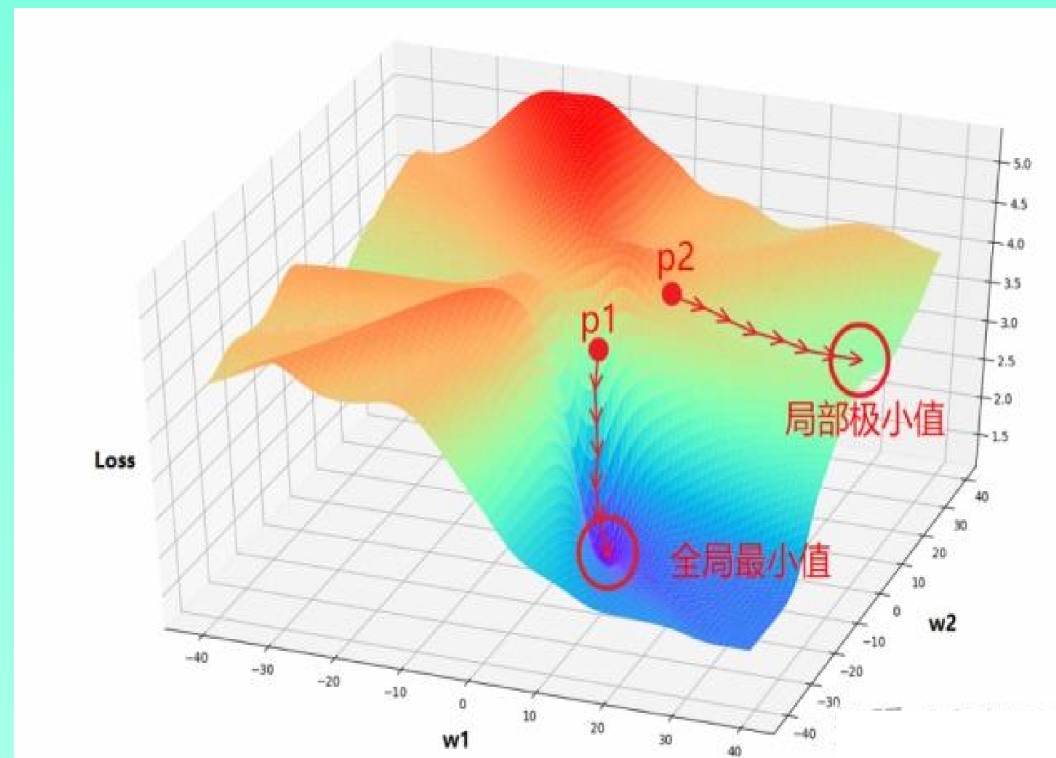
五倍学习率，并逐步减小



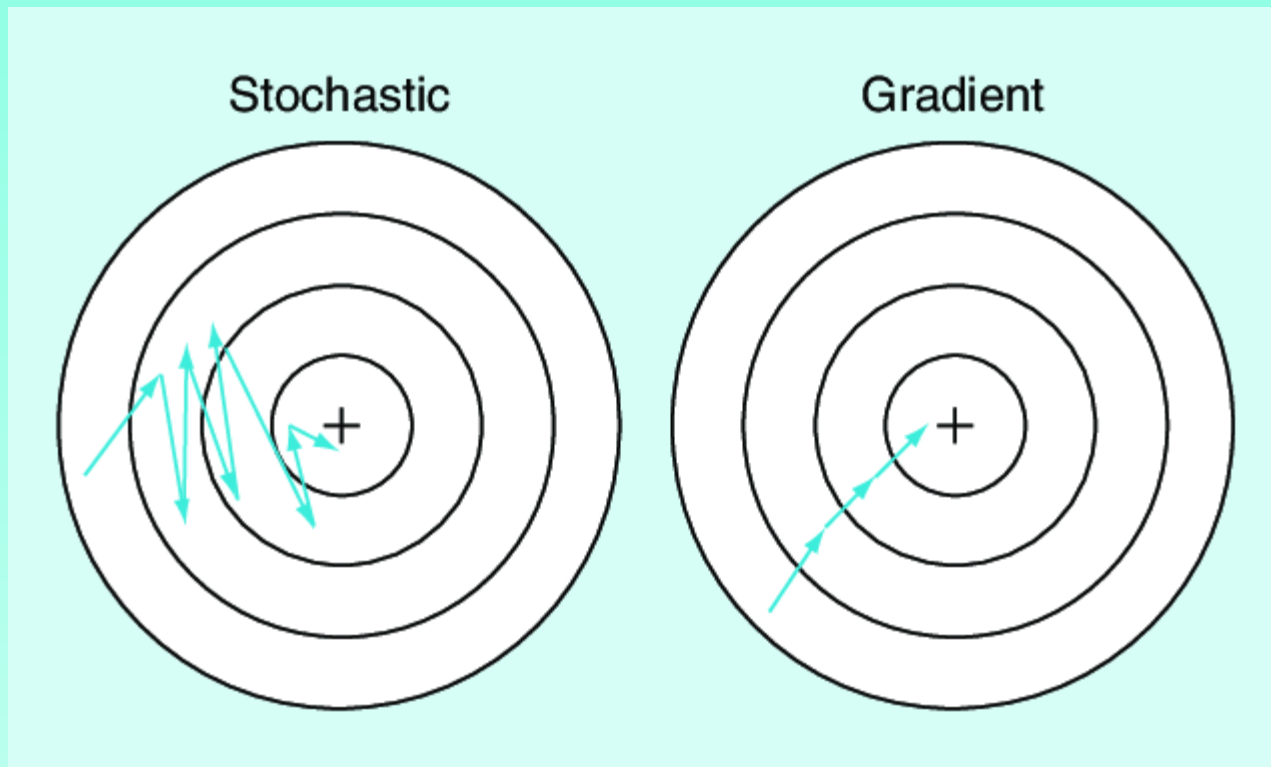
再增大 50%，行为失控，下降过程来回摇摆，
不仅超过了最小值，而且离它越来越远

4.4.3.3 梯度下降的随机近似

- 梯度下降是一种重要的通用学习范型
 - 用于搜索庞大或无限大假设空间 H
 - H 应满足以下条件
 - H 包含连续参数化假设
 - 误差对于这些假设参数可微
- 应用梯度下降法的实际问题
 - 收敛速度缓慢
 - 如果误差面中存在多个局部最小值，则不能保证该过程能够找到全局最小值



- 随机梯度下降(增量梯度下降)
 - 根据每个单独样例的误差增量更新权值，得到近似的梯度下降搜索
 - 注意：如何对表4.1中(批处理梯度下降)训练规则进行修改



Stochastic gradient(*training_examples*, η)

- Training_examples 中每个训练样例形式为序偶 $((x_1, \dots, x_n), t)$ 其中 $(x_1, \dots, x_n)$ 是输入值向量, t 是目标输出值, η 是学习率
- 初始化每个 w_i 为一个很小的随机值
- 在满足终止条件之前, 我们执行
 - 初始化每个 Δw_i 为 0
 - 对每一个 在训练样例中的序偶 $\langle (x_1, \dots, x_n), t \rangle$, 执行
 - 输入实例 $(x_1, \dots, x_n)$ 到线性单元并计算输出 o
 - $\Delta w_i = \eta (t - o) x_i$
 - $w_i = w_i + \Delta w_i$

- 理解随机梯度下降

- 为每个训练样例 $\mathbf{d}$ 创建一个不同的误差函数

$$E_d(\vec{w}) = \frac{1}{2} (t_d - o_d)^2$$

- 在迭代所有训练样例时，这些权重更新的序列给出了对于原误差函数 E 的梯度的合理近似

- $\Delta \mathbf{w}_i = \Delta \mathbf{w}_i + \eta (t - o) \mathbf{x}_i$

- SGD与真实的GD任意接近

- 当学习率足够小时

- 标准(批处理)梯度下降法和随机梯度下降法的关键区别

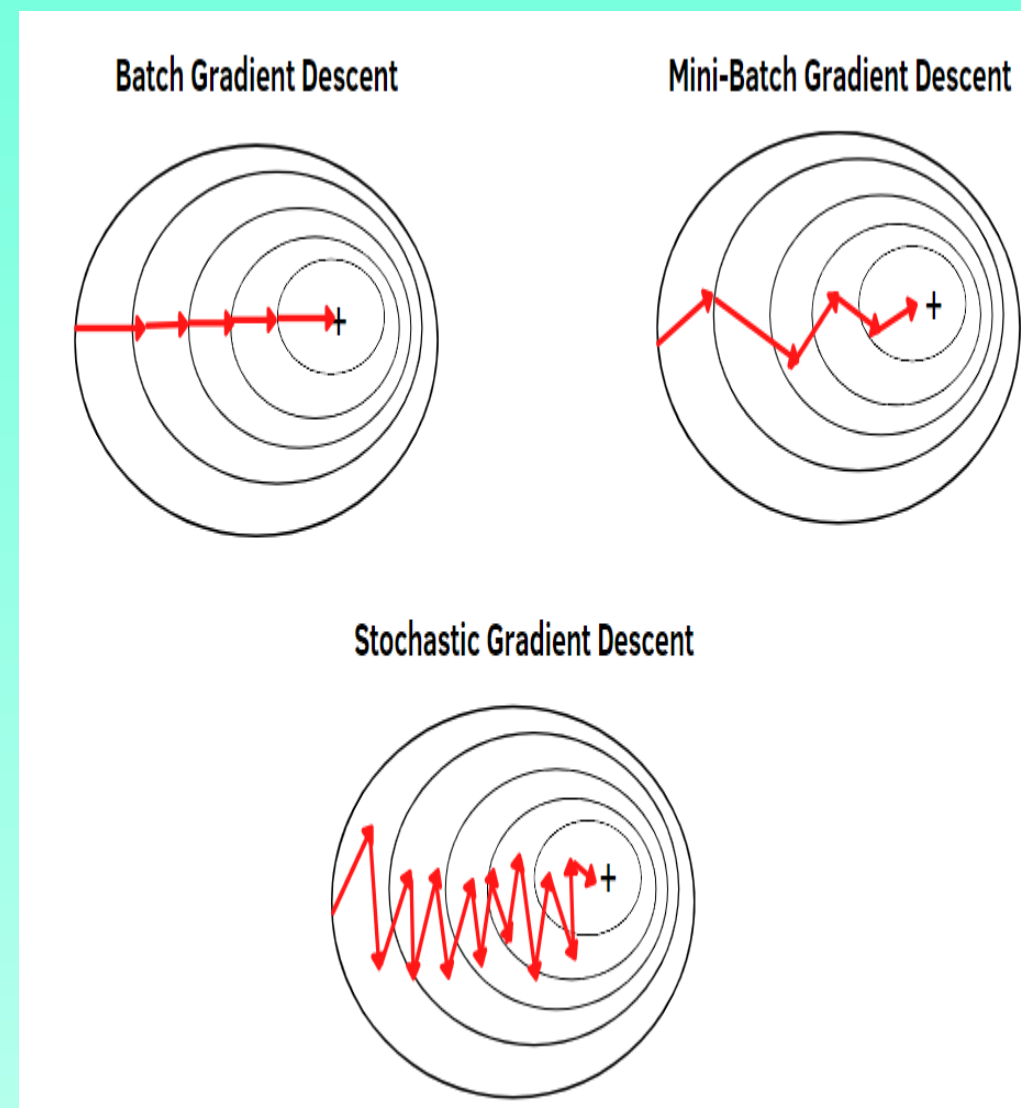
- 书本69页(3点)

- 训练样例的数量
 - 计算量、权值更新的步长
 - 逃离局部极小值的机会

- 随机梯度下降法和标准梯度下降法在实践中都是常用的方法

- 深度学习: mini-Batch Gradient Descent

- epoch: 用全部数据训练一次



- Delta规则讨论
 - Delta规则（增量法则）也称为LMS规则、Adaline规则、windrow-hoff规则
 - 式(4.10)与式(4.4.2)中感知器训练法则的区别(相同?)
 - $\Delta w_i = \eta (t - o) x_i$
 - Delta也可以用来训练阈值感知器单元
 - 如果无阈值输出 o 可以完美拟合目标值，那么阈值输出 o' 也会拟合目标值
 - 即使目标值不能完美匹配，只要线性单元输出 o 与目标值符号一致，阈值 o' 值也能正确匹配目标值
 - Delta法则使线性单元学习到具有输出误差最小的权重，这些权重不一定会使阈值单元输出误差最小。

4.4.4 小结

- 感知器法则和增量(Delta)法则的关键区别
 - 感知器法则根据阈值单元的误差更新权重
 - Delta规则根据线性单元的误差来更新权重
- 收敛属性的差别
 - 假设训练样例是线性可分的，感知器法则可在有限次数的迭代后收敛到分类正确的h
 - Delta规则只收敛于最小误差h，可能需要极长时间，不管训练数据是否线性可分都会收敛

- 学习权向量的第三种算法是线性规划
 - 线性规划是求解线性不等式方程组的一般、有效的方法
 - 这种方法只有在训练样例是线性可分的情况下才有效
 - 这种方法不能扩展到训练多层网络。相比之下，梯度下降法可以很容易地推广到多层网络